

VERERBUNG

OP1 - Die Säulen der Objektorientierten Programmierung

Java

Inhalt

1	Begriffe	2
1.1	Vererbung	2
1.2	Die 4 Sichtbarkeitsstufen in Java	2
1.3	Wichtige Konzepte	2
1.4	Beispiel.....	3
1.5	Polymorphie	4
1.6	UML-Klassendiagramm	4

1 Begriffe

1.1 Vererbung

Eine Klasse (Subklasse) erbt die **Eigenschaften und Methoden** einer anderen Klasse (Superklasse). Damit kann Code **wiederverwendet** und **erweitert** werden.

1.2 Die 4 Sichtbarkeitsstufen in Java

Modifizier	Zugriff innerhalb der Klasse	Zugriff im Paket	Zugriff durch Subklassen	Zugriff außerhalb
<i>private</i>	☒	✗	✗	✗
kein Modifizier = package-private	☒	☒	✗	✗
<i>protected</i>	☒	☒	☒	✗
<i>public</i>	☒	☒	☒	☒

1.3 Wichtige Konzepte

Konzept	Erklärung
<i>super</i>	Zugriff auf die Superklasse (Methoden oder Konstruktor)
Überschreiben (<i>Override</i>)	Subklasse ersetzt Verhalten einer Methode der Superklasse
Konstrukturen	Konstrukturen werden nicht vererbt! Aber <i>super()</i> kann sie aufrufen.

1.4 Beispiel

```
class Fahrzeug {  
    protected int geschwindigkeit = 0;  
  
    public void beschleunigen(int increment) {  
        geschwindigkeit += increment;  
    }  
}  
  
public final class Auto extends Fahrzeug {  
    private int tankinhalt = 100;  
  
    @Override  
    public void beschleunigen(int increment) {  
        if(tankinhalt > 0) {  
            super.beschleunigen(increment);  
            tankinhalt -= increment;  
        } else {  
            System.out.println("Tank leer!");  
        }  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Fahrzeug f = new Auto(); // Polymorphie  
        f.beschleunigen(10);    // nutzt Auto-Version  
    }  
}
```

1.5 Polymorphie

Polymorphie bedeutet: Eine Variable, ein Objekt oder eine Methode kann unterschiedliche Formen annehmen.

Überschreiben (*Override*) = Laufzeit-Polymorphie — Methode verhält sich je nach Objekt anders.

- Überschriebene Methoden müssen die gleiche Signatur haben.
- Sichtbarkeit darf **nicht** eingeschränkt werden (z.B. *public* bleibt *public*).

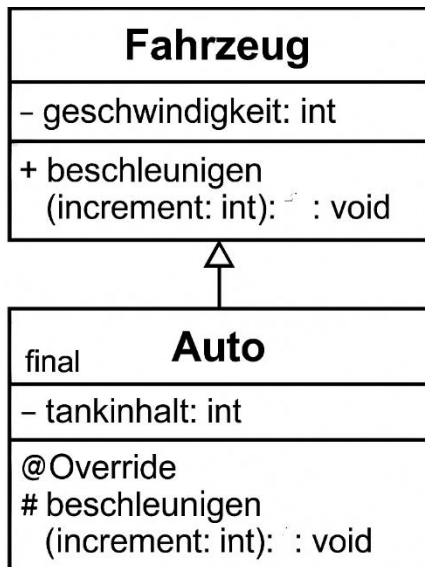
Überladen (*Overload*) = Kompilierzeit-Polymorphie — gleiche Methodennamen, andere Parameter.

```
int addiere(int a, int b) {return a + b;}
```

```
double addiere(double a, double b) {return a + b;}
```

1.6 UML-Klassendiagramm

Symbol	Sichtbarkeit
+	Öffentlich
-	Privat
#	Geschützt (protected)
~	Paketsichtbar



PlantUML

